Dealing with the Elephant in the KV Cache: Video Frame Sampling for Multimodal LLM Inference

Konstantinos Papaioannou
IMDEA Software Institute
Universidad Politécnica de Madrid
konstantinos.papaioannou@imdea.org

Thaleia Dimitra Doudali
IMDEA Software Institute
Madrid, Spain
thaleia.doudali@imdea.org

Abstract

Multimodal Large Language Models (MLLMs) enable rich interactions by processing text alongside images, audio, and video. Among these modalities, video is disproportionately expensive: our characterization shows that a single video can consume up to 5.6 GB of GPU memory, inflating prefill latency and dominating the KV cache. As a result, videos act as an "elephant" in the KV cache, severely degrading performance in multimodal workloads. Existing systems acknowledge this bottleneck but avoid addressing it directly, treating video as the "elephant in the room", or in this case "in the KV Cache". They rely on resource-heavy disaggregation or focus on model-level optimizations, leaving the dominant cost driver (video length) unchanged.

Instead, this paper explores whether the video impact can be reduced at its source by shrinking the input into a few, informative video frames. While reducing the frame count yields major memory and performance benefits, doing so aggressively can significantly hurt inference accuracy. However, smarter frame-selection techniques, such as detecting scene changes or sharpness, can preserve and even improve accuracy. Our evaluation shows that lightweight frame sampling can meaningfully shrink video inputs and tame the "elephant" in the KV cache by reducing memory usage, speeding up inference, and maintaining accuracy. This proof-of-concept prototype establishes the great benefits from directly reducing heavy multimodal inputs, motivating future work in this direction.

1 Introduction

Large Language Models (LLMs) have transformed language applications and are now widely deployed in products like ChatGPT, Gemini, and Copilot [17, 27, 30]. As user interactions increasingly include images, video, and audio, systems are turning to Multimodal Large Language Models (MLLMs) that jointly process these modalities with text, enabling tasks like image reasoning, video understanding, and audio captioning [2, 22, 29]. Emerging "any-to-any" models further expand this paradigm by supporting the generation of multimodal outputs [41].

Multimodal LLMs introduce vision *preprocessing* and *encoding* stages before reaching the LLM backend (Figure 1), substantially expanding the inference pipeline and its associated latency. Text requests, while highly variable in length [6, 33, 37, 42, 49], remain lightweight compared to visual inputs. In contrast, visual inputs, especially *video*, dominate resource usage [42]. For example, our characterization shows that there can be cases where a *single* video can occupy up to 5.6 GB of GPU memory. Therefore, videos can be considered the *"elephants"* in multimodal LLM inference, since they dominate memory usage. When these *"elephants enter the room"* (the workload mix), they significantly degrade overall inference performance, pushing latency far beyond the responsiveness targets of interactive chatbots [31, 35, 50].

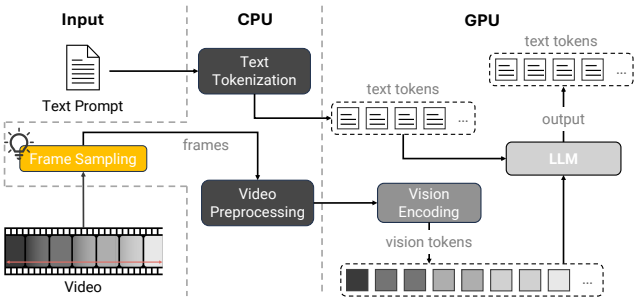


Figure 1. Multimodal LLM (MLLM) Inference Stages.

Recent works acknowledge that multimodal inputs slow down multimodal inference, yet most avoid addressing this issue *directly*. Videos, in particular, are treated as the *"elephant in the room"*, or in this case, the *"elephant in the KV Cache"*. Everyone recognizes their disproportionate impact, but existing systems largely sidestep the problem rather than confronting it head-on. For example, ModServe [32] mitigates the impact of large visual inputs by disaggregating preprocessing, vision encoding, and LLM prefill across abundant cluster resources, effectively bypassing the overheads of multimodal inference rather than reducing them. ServeGen [42] profiles multimodal workloads but treats videos as image bursts, missing their temporal and memory overheads. Other approaches adapt text-only LLM optimizations, such as attention modifications [21] or token-caching schemes [25, 28], without addressing the dominant costs of multimodal inputs.

Current works overlook a *critical detail*: the entire video is still fed into the inference pipeline, leaving little room to reduce its impact. Instead, we ask: *Can we reduce the impact of video directly, at its source, by shrinking the input itself, selecting a few informative frames rather than processing the full sequence?* This shifts the focus from compensating for video inefficiencies to eliminating them before they enter

the MLLM pipeline. However, reducing frames can harm inference accuracy, because it may remove critical information, so the choice and number of frames needs to be carefully designed. Our analysis shows that reducing the number of video frames brings substantial memory and performance benefits, but below a certain threshold it leads to significant accuracy drops. However, we also find that smarter frame-selection techniques, such as ones that detect changes in scenes or sharpness, can preserve, and sometimes even recover, the lost accuracy. Our proof-of-concept system prototype demonstrates that a lightweight Frame Sampling component (Figure 1) can meaningfully shrink video inputs, reduce their memory and latency footprint, and mitigate the "elephant" at its source.

The specific **paper contributions** are:

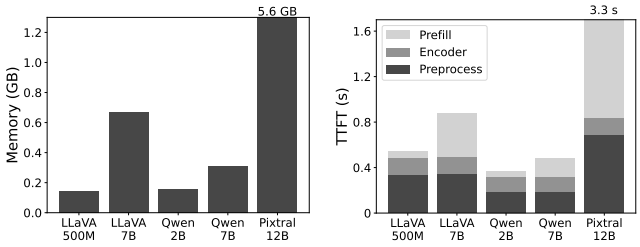
- We quantify a significant performance gap between traditional text-only LLM and multimodal MLLM inference, showing how video inputs inflate memory usage and TTFT (Section 2).
- We explore the design space of frame sampling by evaluating both frame-count reduction and semantics-aware frame selection techniques. This exploration shapes our final proposed approach and serves as an initial proof-of-concept prototype that motivates future work in this direction (Section 3).
- Evaluations shows that our proposed approach effectively narrows the multimodal performance gap, highlighting the potential of addressing the "elephant in the KV cache" directly at its source (Section 4).

2 Motivation

This section first outlines the internal architectural components of MLLMs (Section 2.1). It then characterizes requests with video inputs, both in isolation and mixed into multimodal workloads (Section 2.2), highlighting their distinct performance compared to traditional LLM workloads.

2.1 Multimodal LLM Architecture

Figure 1 illustrates the internal architecture of a multimodal LLM (MLLM). During inference, the model processes video data alongside a textual query. For instance, a video of a busy city street and a question "Which is faster, the white car or the bicycle?". The video is first sampled into frames, preprocessed for the encoder, and transformed into embeddings that capture temporal and spatial semantics in a unified format. Simultaneously, the text is tokenized and mapped to embeddings via a predefined vocabulary. MLLM families vary in their approach to video preprocessing and encoding. Some preprocessors resize videos to a fixed resolution (e.g., LLaVA-OneVision [22]), while others preserve aspect ratios, before slicing frames into patches to prepare them for the encoder. The resulting number of vision tokens per frame typically depends on the frame's resolution. To mitigate computational and memory overhead, some inference engines, such as vLLM [39], *sample a fixed number of frames uniformly* rather than processing the entire sequence.



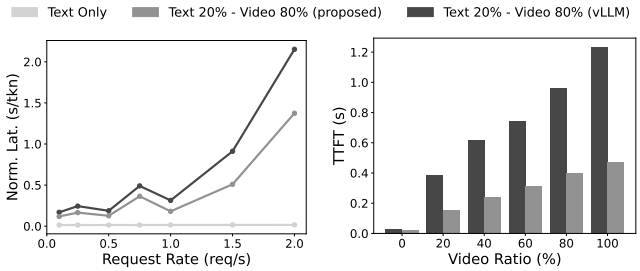
(a) Memory Footprint **(b)** TTFT Breakdown

Figure 2. Average memory footprint and TTFT for requests that contain video inputs across different MLLMs.

At its core, the MLLM leverages a transformer-based LLM backbone to process these embeddings across two distinct phases: prefill (or prompt) and decode [44]. In the prefill phase, the input prompt is processed either all at once or in large chunks if chunked-prefill is enabled [3, 4]. Subsequently, the decode phase generates text auto-regressively, one token at a time. While the decode phase is less compute-intensive than prefill, it is highly memory-constrained due to the KV Cache, which stores the initial prompt and previously generated tokens to prevent redundant computation. However, the KV Cache size can scale significantly, often reaching GPU memory limits [16, 19, 23].

2.2 Multimodal Workload Characterization

2.2.1 In Isolation. To understand the impact of video prompts, we characterize their memory footprint (KV Cache usage) and inference latency (time-to-first-token TTFT) in isolation, as shown in Figure 2. The experimental setup is detailed in Section 4.1. We find that both memory usage and TTFT depend strongly on the model family and size. For example, LLaVA-500M requires roughly 0.15 GB of KV Cache space per video, on average, which increases to about 0.65 GB for LLaVA-7B; Qwen-2B and Qwen-7B exhibit a similar but smaller-scale trend. Pixtral is the most memory-intensive model, with a single video reaching an average of 5.6 GB. Overall, memory behavior mirrors TTFT: although preprocessing and vision encoding times are comparable across models, LLM prefill time varies substantially. Pixtral has the longest prefill time, consistent with its much larger memory footprint, resulting in a TTFT of 3.3 seconds. This time is far beyond the responsiveness targets of commercial platforms for interactive applications such as chatbots [31, 35, 50]. In conclusion, video prompts place significantly heavy demands on memory resources, and this pressure directly manifests in inference performance. Therefore, they can be perceived as the *"elephant" in the KV cache.*



(a) Increasing Request Rate **(b)** Increasing Video Ratio

Figure 3. Performance of multimodal workloads under increasing request rates and ratio of videos in the workload mix.

2.2.2 In Workload. Next, we examine how the presence of an "elephant" request disturbs multimodal workload inference and how this behavior compares to traditional LLM inference on text-only prompts. Figure 3a reports normalized latency (seconds per token) under increasing request rates for a multimodal workload containing 80% text-only prompts and 20% video prompts, compared to a 100% text-only baseline, for LLaVA-7B. Section 4 will evaluate the proposed approach, for now we focus on the vLLM baseline. While traditional text workloads can sustain high request rates with minimal impact on normalized latency, multimodal workloads degrade sharply: normalized latency rises steeply as request rate increases, reaching values as high as 2 seconds per token, far from acceptable interactive performance [31, 35, 50]. In addition, Figure 3b focuses exclusively on multimodal workloads and varies the fraction of prompts with video in the workload. As the ratio of videos increases, TTFT latency worsens proportionally, illustrating how the elevated memory footprint and prefill time of video requests slow down the entire system. In conclusion, even a modest fraction of video prompts can significantly disrupt inference performance, underscoring their role as a dominant bottleneck in multimodal inference.

Takeaways: Videos are the *"elephants"* in MLLM inference: they dominate memory usage, and when these *"elephants enter the room"* (the workload mix), they significantly degrade overall inference performance, pushing latency far beyond the responsiveness targets of popular chatbot systems.

3 Approach Exploration

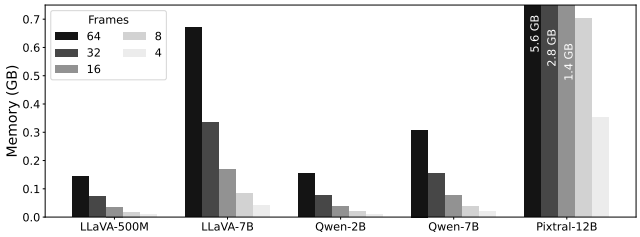
Recent works on multimodal LLM inference often treat multimodal inputs (especially video) as the *"elephant in the room"*. The performance slowdown introduced by video is acknowledged, yet many systems avoid addressing it *directly*. For example, ModServe [32] assumes access to large-scale clusters with abundant resources and mitigates bottlenecks by disaggregating preprocessing, vision encoding, and LLM prefill into separate stages, effectively bypassing the core inefficiencies of video inputs rather than reducing them. ServeGen [42] characterizes production-scale multimodal serving workloads but focuses primarily on language and image–text models, treating videos as simple bursts of images and overlooking their additional temporal and memory overheads. Other systems apply techniques traditionally successful in LLM inference, such as modifying the attention mechanism to reduce computation [21] or caching only the most relevant tokens [25, 28], without explicitly targeting the dominant costs introduced by video frames.

Together, these approaches fail to capture the true bottlenecks that video introduces into multimodal LLM inference and *miss the opportunity* to improve inference by "acknowledging the elephant" and directly reducing its impact. This gap motivates our exploration. We aim to build a system component, as shown in Figure 1, that operates *before* the video preprocessing of the multimodal LLM inference pipeline and *reduces the overhead of video inputs directly at their source*. This is similar to the strategy already used in vLLM [39] that has a Frame Sampling component, which takes the input video and samples a fixed number of frames uniformly rather than processing the entire video. Inspired by this direction, we explore approaches that further reduce the computational and memory overheads introduced by video inputs. Our *goal* is to identify techniques that not only reduce the number of frames entering the pipeline but also improve the quality of the selected frames, *ultimately minimizing the "elephant" at its source*, while preserving high inference accuracy.

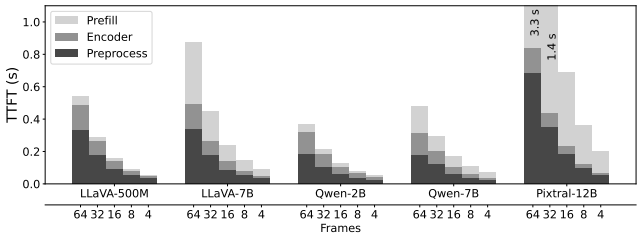
3.1 Reducing Frame Count

To begin with, we examine the most naive and straightforward strategy: simply reducing the total number of frames presented to the model. We start by sampling 64 frames and experiment with progressively halving the number of frames, evaluating configurations from 64 down to 4 frames. Figure 4 summarizes the impact of this reduction on (a) memory footprint, (b) TTFT latency, and (c) inference accuracy, on average, across different multimodal model families and sizes. We observe that memory usage scales mostly linearly with the number of frames: halving the frames nearly halves the KV Cache footprint. For example, Pixtral-12B drops from 5.6 GB at 64 frames down to 2.8 GB at 32. This reduction is mirrored in TTFT, where Pixtral-12B improves from 3.3 s to 1.4 s, moving much closer to interactive latency targets [31, 35, 50]. This reduction in memory and latency highlight the *substantial opportunity* that arises from just reducing frame count.

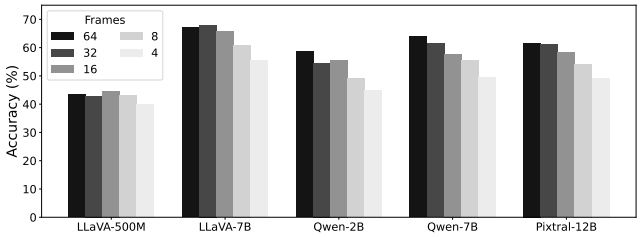
However, aggressively reducing video frames risks discarding important visual information. We therefore examine the effect on inference accuracy. Small models like LLaVA-500M retain around 40% accuracy regardless of frame count due to their limited capabilities. Larger models show notable drops (up to 15%) when reduced between 16 and 4 frames.



(a) Memory usage (KV Cache)



(b) TTFT breakdown



(c) Inference Accuracy

Figure 4. Inference performance and accuracy across MLLMs for decreasing frame count.

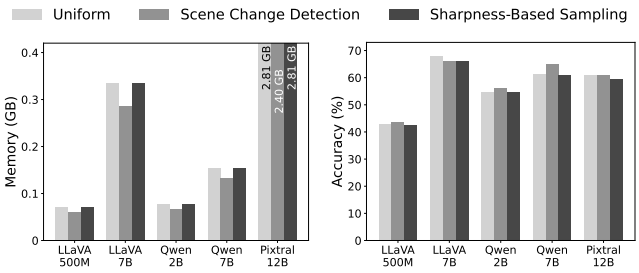
Interestingly, accuracy at 32 frames remains similar or less just by 1-4% for some models like Qwen, compared to the 64-frame baseline.

Takeaways: The naive approach of directly reducing the number of frames yields memory and TTFT improvements *proportional* to the reduction in frame count. Inference accuracy remains comparable down to 32 frames but degrades significantly beyond that point.

3.2 Selecting More Informative Frames

While reducing the number of frames provides clear performance and resource efficiency benefits, it can introduce accuracy losses. A natural question arises: *Can we recover this lost accuracy by selecting more informative frames instead of simply fewer frames?* Rather than treating all frames as equally useful, as done in uniform sampling, we investigate whether choosing frames that capture meaningful visual changes or higher-quality content can preserve inference accuracy. To this end, we explore the following widely used video frame sampling techniques:

- **Uniform (vLLM)**: Selects a fixed number of frames at regular intervals across the video duration and is used as the default strategy in inference engines such as vLLM [39]. It is computationally inexpensive and simple to configure, but treats all frames as equally informative and ignores the temporal dynamics of the video.
- **Scene Change Detection (SCD) [11]**: Identifies significant visual transitions ("scene cuts") and selects frames around these points. By capturing moments where the visual content changes meaningfully, SCD aims to preserve semantic structure. This approach is commonly used in surveillance, video editing, and industrial monitoring applications.
- **Sharpness-Based Sampling (SBS) [7]**: Prioritizes frames with high quality by filtering out blurry or low-fidelity ones. This method is widely applied in video compression, denoising, and ML dataset construction, and helps ensure that only visually reliable frames enter the model.



(a) Memory usage (KV Cache) **(b)** Inference Accuracy

Figure 5. Average memory per video and accuracy across MLLMs and different frame sampling techniques.

Next, we focus on the 32-frame configuration, since Section 3.1 showed that it provides substantial efficiency and performance gains while mostly maintaining accuracy. Figure 4 summarizes how different frame selection techniques affect (a) the memory footprint of a video and (b) inference accuracy, on average, across the multimodal models. We observe that Scene Change Detection consistently uses less memory across all models. For example, reducing Pixtral-12B's usage by 0.4 GB and LLaVA-7B's by 0.05 GB compared to uniform sampling. Our analysis shows that this improvement is also mirrored in TTFT with lower latencies, respectively. The reason this happens is that Scene Change Detection selects frames only when a significant visual transition occurs. As a result, fewer frames are sampled compared to uniform or sharpness-based methods, which always return a fixed number of frames and therefore exhibit similar memory and latency behavior. What is particularly interesting is that *accuracy can even improve* in some cases when using Scene Change Detection. For example, Qwen-7B sees a 5% accuracy increase, likely because it selects more informative frames, whereas uniform sampling may miss frames containing critical visual information.

Figure 6 illustrates this effect. It shows how the three techniques (rows) select different frames from the same video. In this example, the prompt asks: "Which color of clothing is

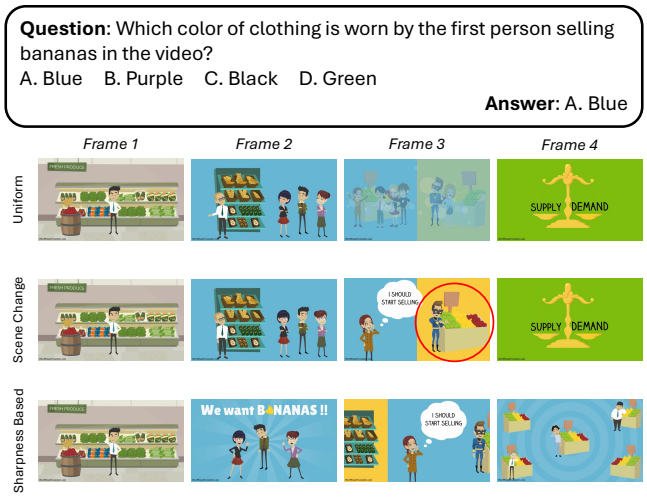
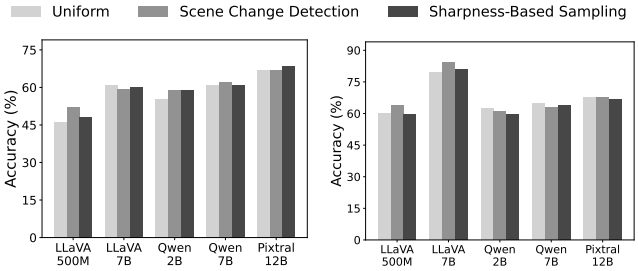


Figure 6. Example of the different frames selected across techniques.



(a) MMBench-Video (Q&A) **(b)** TempCompass (Captioning)

Figure 7. Inference accuracy across different benchmarks.

worn by the first person selling bananas in the video?" The correct answer is *blue*, visible only in the third frame selected by Scene Change Detection. Uniform sampling fails because its third frame is blurry due to a transition, obscuring the relevant details. Sharpness-based sampling selects a clear frame, but it appears just before the one revealing the seller wearing blue, thus missing the critical information. As a result, Scene Change Detection answers correctly, while the other two techniques fail to do so.

Generalizability across benchmarks. Figure 7 reports accuracy across models and sampling techniques for two additional benchmarks, whose details are provided in Section 4.1. We observe that although the three techniques often achieve comparable accuracy, there are several cases where Scene Change Detection offers clear improvements, like for LLaVA-500M and LLaVA-7B. This further validates and highlights the *benefit of selecting more informative frames.*

Sampling overheads. Uniform sampling and Scene Change Detection for 32 frames incur similar overheads of roughly 1.6 seconds, on average, per video. Sharpness-Based Sampling is more expensive at around 4.2 seconds, due to its per-frame sharpness evaluation. It is important to note that these times are not reflected in TTFT, because they occur *before* the multimodal LLM pipeline begins. This is consistent with the default behavior of vLLM with uniform sampling, where the sampling step is performed prior to video preprocessing, as shown in Figure 1. In conclusion, Scene Change Detection provides higher-quality frame selection *without introducing additional overhead* compared to vLLM's default uniform sampling.

Takeaways: Scene Change Detection selects more informative frames, which can even improve accuracy. It often selects fewer frames than uniform sampling, thereby reducing memory usage and TTFT. Most importantly, it achieves these benefits without incurring any additional overhead compared to the uniform sampling strategy used in current systems.

3.3 Proposed Approach

Informed by the observations in Sections 3.1 and 3.2, we propose a simple yet effective approach that *combines a reduced frame count with more informative frame selection.* Concretely, our system component **samples 32 frames**, which is the lowest frame count that preserves accuracy across models, using **Scene Change Detection (SCD)** to prioritize semantically meaningful transitions. This combination retains model accuracy while significantly reducing memory footprint and TTFT. Experimental evaluation (Section 4) quantifies these benefits and demonstrates how the proposed approach directly addresses and mitigates the impact of the "elephant in the room".

Future Work. Although effective, this design serves only as an initial system prototype to establish and quantify the benefits of the proposed direction. Future work will build on these insights and explore more dynamic, accuracy-aware input adaptation mechanisms that adjust the video frame count and selection strategy based on parameters like the model family and size, accuracy guarantees, performance objectives, workload conditions and system load.

4 Evaluation

This section evaluates the approach proposed in Section 3.3 and demonastrates how it closes the performance gap quantified in Section 2.2.2, using the experimental setup below.

4.1 Experimental Setup

Testbed. We experiment on a server with native hardware that includes one NVIDIA A100 GPU with 40GB memory, two AMD EPYC 7313 16-Core processors (32 threads) and 256GB of host DRAM memory.

Models. Table 1 shows the state-of-the-art multimodal models used in our evaluation, grouped by family and size, listing their parameter count alongside the internal vision encoder and LLM backend. LLaVA refers to LLaVA-OneVision [22],

Abbreviation	Vision Encoder	LLM Backend
LLaVA-500M	SigLIP (400M)	Qwen2 (500M)
LLaVA-7B	SigLIP (400M)	Qwen2 (7B)
Qwen-2B	Custom ViT (675M)	Qwen2 (1.5B)
Qwen-7B	Custom ViT (675M)	Qwen2 (7B)
Pixtral-12B	Pixtral-ViT (400M)	Mistral NeMo (12B)

Table 1. Multimodal models (MLLMs) used for evaluation.

Qwen to Qwen2-VL [40] from Alibaba Cloud, and Pixtral [5] from Mistral AI. We deploy our models using vLLM [39], which natively supports all of them. For the Pixtral model, we process videos as sequences of images (frames) since this model do not natively support video inputs.

Benchmarks. We experiment with the following video benchmarks. Video-MME [15] focuses on multi-choice video analysis, MMBench-Video [13] provides open-ended question answering for video understanding, and TempCompass [24] assesses fine-grained temporal perception and captioning. For the text-only workload characterization in Figure 3, we use conversations from ShareGPT [33]. Unless otherwise specified, we experiment with Video-MME.

Evaluation Metrics. Latency is measured using Time to First Token (TTFT) and Normalized Latency, defined as the end-to-end request latency divided by the output length [20, 44]. Memory Footprint is approximated as the peak KV Cache usage by multiplying the number of generated tokens by the model-specific per-token memory cost. For Accuracy, we report the percentage of correct answers on multiple-choice tasks, and use Flow-Judge-v0.1 [14] to label model outputs as correct or incorrect for open-ended tasks.

Implementation. We use vLLM [39] (v0.8.4) with the V1 engine and the chunked prefill optimization [38]. We extend vLLM's metrics collection to precisely capture multimodal latency and memory usage, and augment the `VideoAsset` class with scene-change detection and sharpness-based sampling for the video frames.

4.2 Bridging the Performance Gap

We revisit Figure 3a, which showed the large performance gap between text-only and multimodal workloads under vLLM's 64-frame uniform sampling. With our approach, this gap narrows substantially. Using 32 frames selected via Scene Change Detection reduces normalized latency by up to 36% at high request rates, keeping it at or below 1 second across the board. This allows multimodal inference to match the responsiveness targets for interactive applications such as chatbots [31, 35, 50]. Figure 3b further shows that, regardless of the number of videos in the workload, TTFT decreases by up to 60% due to reduced prefill time and scheduling delays enabled by the lower memory footprint.

Takeaways: Our proposed approach substantially narrows the gap between text-only and multimodal inference by lowering normalized latency to within interactive thresholds and reducing TTFT by up to 60%.

5 Related Work

The beginning of Section 3 already discusses the limitations of the most relevant works, who acknowledge but do not directly mitigate the substantial overhead introduced by video inputs. In general, these systems either rely on large-scale resource disaggregation, as in ModServe [32], or focus on workload characterization without modeling video-specific temporal and memory costs, as in ServeGen [42]. Recent MLLM-oriented techniques repurpose text-only LLM optimizations to vision by operating after video frames have already been converted into tokens. These approaches reduce cost by selectively pruning, scoring, or restructuring vision tokens, including FastV [8], AdaptInfer [46], SparseVLM [47], PACT [12], GlimpsePrune [45], and Inf-MLLM [28].

The above techniques, build on optimizations already established to be effective for traditional LLM inference. KV cache quantization methods compress keys and values via low-precision formats or asymmetric schemes, including FlexGen [34], KVQuant [18], KIVI [26], LLM.int8() [10], and KVSink [36]. Beyond quantization, cache-eviction and token-reduction techniques prune or merge uninformative tokens to meet cache budgets: StreamingLLM [43] and H2O [48] maintain only recent or high-attention tokens, while NACL [9] and Keyformer [1] propose optimized eviction policies.

Different from all works above, our approach reduces overhead *before* multimodal processing begins, shrinking the video input itself and preventing excessive token and KV Cache growth at the source.

6 Summary

This paper presents a lightweight video frame sampling component that reduces multimodal inference cost by shrinking video inputs before they enter the inference pipeline. By selecting only a few informative frames instead of processing the full video sequence, we show that memory usage and TTFT can be drastically reduced while preserving accuracy, enabling multimodal inference that meets the responsiveness targets of popular chatbots. Our proof-of-concept prototype highlights the potential of reducing multimodal inputs at the source and motivates future work in this direction.

Acknowledgments

We thank the anonymous reviewers for their constructive feedback. We would also like to thank Ioannis Dalianis for his contributions during his internship at IMDEA Software Institute, which helped shape the inspiration for and some of the preliminary results of this work.

This work has been partially funded by the Madrid Regional Government (César Nombela grant 2024-T1/COM-31302) and partially supported by Comunidad de Madrid as part of the DATIA project, co-funded by FEDER Funds of the European Union. It is also part of the grant PID2022-142290OB-I00, funded by MCIN/AEI/10.13039/501100011033/FEDER, UE and the grant CEX2024-001471-M funded by MICIU/AEI/10.13039/501100011033.

References

[1] Muhammad Adnan, Akhil Arunkumar, Gaurav Jain, Prashant J. Nair, Ilya Soloveychik, and Purushotham Kamath. 2024. Keyformer: KV Cache reduction through key tokens selection for Efficient Generative Inference. In *Proceedings of Machine Learning and Systems*, P. Gibbons, G. Pekhimenko, and C. De Sa (Eds.), Vol. 6. 114–127. https://proceedings.mlsys.org/paper_files/paper/2024/file/48fecef47b19fe501d27d338b6d52582-Paper-Conference.pdf

[2] Armen Aghajanyan, Sony Theakanath, Lili Yu, and Luke Zettlemoyer. 2024. Introducing CM3leon, a more efficient, state-of-the-art generative model for text and images. https://ai.meta.com/blog/generative-ai-text-images-cm3leon/.

[3] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2025. Taming throughput-latency tradeoff in LLM inference with sarathi-serve. In *Proceedings of the 18th USENIX Conference on Operating Systems Design and Implementation* (Santa Clara, CA, USA) *(OSDI'24)*. USENIX Association, USA, Article 7, 18 pages.

[4] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, and Ramachandran Ramjee. 2023. SARATHI: Efficient LLM Inference by Piggybacking Decodes with Chunked Prefills. arXiv:2308.16369 [cs.LG] https://arxiv.org/abs/2308.16369

[5] Pravesh Agrawal, Szymon Antoniak, Emma Bou Hanna, Baptiste Bout, Devendra Chaplot, Jessica Chudnovsky, Diogo Costa, Baudouin De Monicault, Saurabh Garg, Theophile Gervet, Soham Ghosh, Amélie Héliou, Paul Jacob, Albert Q. Jiang, Kartik Khandelwal, Timothée Lacroix, Guillaume Lample, Diego Las Casas, Thibaut Lavril, Teven Le Scao, Andy Lo, William Marshall, Louis Martin, Arthur Mensch, Pavankumar Muddireddy, Valera Nemychnikova, Marie Pellat, Patrick Von Platen, Nikhil Raghuraman, Baptiste Rozière, Alexandre Sablayrolles, Lucile Saulnier, Romain Sauvestre, Wendy Shang, Roman Soletskyi, Lawrence Stewart, Pierre Stock, Joachim Studnia, Sandeep Subramanian, Sagar Vaze, Thomas Wang, and Sophia Yang. 2024. Pixtral 12B. arXiv:2410.07073 [cs.CV] https://arxiv.org/abs/2410.07073

[6] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. 2024. LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding. arXiv:2308.14508 [cs.CL] https://arxiv.org/abs/2308.14508

[7] Raghav Bansal, Gaurav Raj, and Tanupriya Choudhury. 2016. Blur image detection using Laplacian operator and Open-CV. In *2016 International Conference System Modeling & Advancement in Research Trends (SMART)*. 63–67. https://doi.org/10.1109/SYSMART.2016.7894491

[8] Liang Chen, Haozhe Zhao, Tianyu Liu, Shuai Bai, Junyang Lin, Chang Zhou, and Baobao Chang. 2024. An Image is Worth 1/2 Tokens After Layer 2: Plug-and-Play Inference Acceleration for Large Vision-Language Models. arXiv:2403.06764 [cs.CV]

[9] Yilong Chen, Guoxia Wang, Junyuan Shang, Shiyao Cui, Zhenyu Zhang, Tingwen Liu, Shuohuan Wang, Yu Sun, Dianhai Yu, and Hua Wu. 2024. NACL: A General and Effective KV Cache Eviction Framework for LLM at Inference Time. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 7913–7926. https://doi.org/10.18653/v1/2024.acl-long.428

[10] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. 2022. GPT3.int8(): 8-bit Matrix Multiplication for Transformers at Scale. In *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh (Eds.), Vol. 35. Curran Associates, Inc., 30318–30332. https://proceedings.neurips.cc/paper_files/paper/2022/file/c3ba4962c05c49636d4c6206a97e9c8a-Paper-Conference.pdf

[11] PySceneDetect Developers. 2026. PySceneDetect v0.6.1 API — ContentDetector. https://www.scenedetect.com/docs/0.6.1/api/detectors.html#module-scenedetect.detectors.content_detector.

[12] Mohamed Dhouib, Davide Buscaldi, Sonia Vanier, and Aymen Shabou. 2025. PACT: Pruning and Clustering-Based Token Reduction for Faster Visual Language Models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 14582–14592. https://doi.org/10.1109/CVPR52734.2025.01359

[13] Xinyu Fang, Kangrui Mao, Haodong Duan, Xiangyu Zhao, Yining Li, Dahua Lin, and Kai Chen. 2024. MMBench-Video: A Long-Form Multi-Shot Benchmark for Holistic Video Understanding. *arXiv preprint arXiv:2406.14515* (2024).

[14] Flow AI. 2024. Flow Judge: An Open Small Language Model for LLM System Evaluations. https://flow-ai.com/blog/flow-judge.

[15] Chaoyou Fu, Yuhan Dai, Yongdong Luo, Lei Li, Shuhuai Ren, Renrui Zhang, Zihan Wang, Chenyu Zhou, Yunhang Shen, Mengdan Zhang, et al. 2025. Video-mme: The first-ever comprehensive evaluation benchmark of multi-modal llms in video analysis. In *CVPR*.

[16] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. 2025. Cost-efficient large language model serving for multi-turn conversations with CachedAttention. In *Proceedings of the 2024 USENIX Conference on Usenix Annual Technical Conference* (Santa Clara, CA, USA) *(USENIX ATC'24)*. USENIX Association, USA, Article 7, 16 pages.

[17] Google. 2024. Gemini Google. https://gemini.google/about/.

[18] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W. Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. 2024. KVQuant: towards 10 million context length LLM inference with KV cache quantization. In *Proceedings of the 38th International Conference on Neural Information Processing Systems* (Vancouver, BC, Canada) *(NIPS '24)*. Curran Associates Inc., Red Hook, NY, USA, Article 40, 34 pages.

[19] Chao Jin, Zili Zhang, Xuanlin Jiang, Fangyue Liu, Xin Liu, Xuanzhe Liu, and Xin Jin. 2024. RAGCache: Efficient Knowledge Caching for Retrieval-Augmented Generation. arXiv:2404.12457 [cs.DC] https://arxiv.org/abs/2404.12457

[20] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles* (Koblenz, Germany) *(SOSP '23)*. Association for Computing Machinery, New York, NY, USA, 611–626. https://doi.org/10.1145/3600006.3613165

[21] Zhihang Lin, Mingbao Lin, Luxi Lin, and Rongrong Ji. 2025. Boosting Multimodal Large Language Models with Visual Tokens Withdrawal for Rapid Inference. arXiv:2405.05803 [cs.CV] https://arxiv.org/abs/2405.05803

[22] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. 2024. Visual instruction tuning. In *Proceedings of the 37th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) *(NIPS '23)*. Curran Associates Inc., Red Hook, NY, USA, Article 1516, 25 pages.

[23] Yuhan Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, Michael Maire, Henry Hoffmann, Ari Holtzman, and

Junchen Jiang. 2024. CacheGen: KV Cache Compression and Streaming for Fast Large Language Model Serving. In *Proceedings of the ACM SIGCOMM 2024 Conference* (Sydney, NSW, Australia) *(ACM SIGCOMM '24)*. Association for Computing Machinery, New York, NY, USA, 38–56. https://doi.org/10.1145/3651890.3672274

[24] Yuanxin Liu, Shicheng Li, Yi Liu, Yuxiang Wang, Shuhuai Ren, Lei Li, Sishuo Chen, Xu Sun, and Lu Hou. 2024. TempCompass: Do Video LLMs Really Understand Videos? arXiv:2403.00476 [cs.CV] https://arxiv.org/abs/2403.00476

[25] Zuyan Liu, Benlin Liu, Jiahui Wang, Yuhao Dong, Guangyi Chen, Yongming Rao, Ranjay Krishna, and Jiwen Lu. 2024. Efficient Inference of Vision Instruction-Following Models with Elastic Cache. arXiv:2407.18121 [cs.CV] https://arxiv.org/abs/2407.18121

[26] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. 2024. KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache. *arXiv preprint arXiv:2402.02750* (2024).

[27] Microsoft. 2025. Microsoft 365 Copilot. https://adoption.microsoft.com/en-us/copilot/.

[28] Zhenyu Ning, Jieru Zhao, Qihao Jin, Wenchao Ding, and Minyi Guo. 2024. Inf-MLLM: Efficient Streaming Inference of Multimodal Large Language Models on a Single GPU. arXiv:2409.09086 [cs.LG] https://arxiv.org/abs/2409.09086

[29] OpenAI. 2024. GPT-4 | OpenAI. https://openai.com/index/gpt-4/.

[30] OpenAI. 2025. ChatGPT. https://chatgpt.com/overview/.

[31] OpenAI. 2025. ChatGPT Priority Processing. https://openai.com/api-priority-processing/.

[32] Haoran Qiu, Anish Biswas, Zihan Zhao, Jayashree Mohan, Alind Khare, Esha Choukse, Íñigo Goiri, Zeyu Zhang, Haiying Shen, Chetan Bansal, Ramachandran Ramjee, and Rodrigo Fonseca. 2025. ModServe: Modality- and Stage-Aware Resource Disaggregation for Scalable Multimodal Model Serving. arXiv:2502.00937 [cs.DC] https://arxiv.org/abs/2502.00937

[33] ShareGPT. 2024. ShareGPT Platform. https://sharegpt.com/.

[34] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. 2023. FlexGen: high-throughput generative inference of large language models with a single GPU. In *Proceedings of the 40th International Conference on Machine Learning* (Honolulu, Hawaii, USA) *(ICML'23)*. JMLR.org, Article 1288, 23 pages.

[35] Jovan Stojkovic, Chaojie Zhang, Íñigo Goiri, Josep Torrellas, and Esha Choukse. 2024. DynamoLLM: Designing LLM Inference Clusters for Performance and Energy Efficiency. arXiv:2408.00741 [cs.AI] https://arxiv.org/abs/2408.00741

[36] Zunhai Su and Kehong Yuan. 2025. KVSink: Understanding and Enhancing the Preservation of Attention Sinks in KV Cache Quantization for LLMs. arXiv:2508.04257 [cs.CL] https://arxiv.org/abs/2508.04257

[37] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. 2023. Stanford Alpaca: An Instruction-following LLaMA model. https://github.com/tatsu-lab/stanford_alpaca.

[38] vLLM. 2024. vLLM - Chunked Prefill. https://docs.vllm.ai/en/latest/performance/optimization.html#chunked-prefill.

[39] vLLM Team. 2025. vLLM: Easy, Fast, and Cheap LLM Serving with PagedAttention. https://vllm.ai. Accessed: 2025-01-01.

[40] Peng Wang, Shuai Bai, Sinan Tan, Shijie Wang, Zhihao Fan, Jinze Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Yang Fan, Kai Dang, Mengfei Du, Xuancheng Ren, Rui Men, Dayiheng Liu, Chang Zhou, Jingren Zhou, and Junyang Lin. 2024. Qwen2-VL: Enhancing Vision-Language Model's Perception of the World at Any Resolution. arXiv:2409.12191 [cs.CV] https://arxiv.org/abs/2409.12191

[41] Shengqiong Wu, Hao Fei, Leigang Qu, Wei Ji, and Tat-Seng Chua. 2024. NExT-GPT: Any-to-Any Multimodal LLM. In *Proceedings of the International Conference on Machine Learning*. 53366–53397.

[42] Yuxing Xiang, Xue Li, Kun Qian, Wenyuan Yu, Ennan Zhai, and Xin Jin. 2025. ServeGen: Workload Characterization and Generation of Large Language Model Serving in Production. arXiv:2505.09999 [cs.DC] https://arxiv.org/abs/2505.09999

[43] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2023. Efficient Streaming Language Models with Attention Sinks. *arXiv* (2023).

[44] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. USENIX Association, Carlsbad, CA, 521–538. https://www.usenix.org/conference/osdi22/presentation/yu

[45] Quan-Sheng Zeng, Yunheng Li, Qilong Wang, Peng-Tao Jiang, Zuxuan Wu, Ming-Ming Cheng, and Qibin Hou. 2025. A Glimpse to Compress: Dynamic Visual Token Pruning for Large Vision-Language Models. arXiv:2508.01548 [cs.CV] https://arxiv.org/abs/2508.01548

[46] Weichen Zhang, Zhui Zhu, Ningbo Li, Shilong Tao, Kebin Liu, and Yunhao Liu. 2026. AdaptInfer: Adaptive Token Pruning for Vision-Language Model Inference with Dynamical Text Guidance. arXiv:2508.06084 [cs.CV] https://arxiv.org/abs/2508.06084

[47] Yuan Zhang, Chun-Kai Fan, Junpeng Ma, Wenzhao Zheng, Tao Huang, Kuan Cheng, Denis A Gudovskiy, Tomoyuki Okuno, Yohei Nakata, Kurt Keutzer, and Shanghang Zhang. 2025. SparseVLM: Visual Token Sparsification for Efficient Vision-Language Model Inference. In *Forty-second International Conference on Machine Learning*. https://openreview.net/forum?id=80faIPZ67S

[48] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. 2024. H2O: heavy-hitter oracle for efficient generative inference of large language models. In *Proceedings of the 37th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) *(NIPS '23)*. Curran Associates Inc., Red Hook, NY, USA, Article 1506, 50 pages.

[49] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Tianle Li, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zhuohan Li, Zi Lin, Eric. P Xing, Joseph E. Gonzalez, Ion Stoica, and Hao Zhang. 2023. LMSYS-Chat-1M: A Large-Scale Real-World LLM Conversation Dataset. arXiv:2309.11998 [cs.CL]

[50] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: disaggregating prefill and decoding for goodput-optimized large language model serving. In *Proceedings of the 18th USENIX Conference on Operating Systems Design and Implementation* (Santa Clara, CA, USA) *(OSDI'24)*. USENIX Association, USA, Article 11, 18 pages.